

ポインタ同士の引き算 — 差は「要素いくつつ分か」

今日のゴール

`p - q`（ポインタ同士の引き算）を、C の規格どおり要素いくつつ分離れているかを返すように直す。

1 この回の位置づけ

- 意味論発展シリーズの第 3 回（選択制）。前提はコマ 10（ポインタ演算）
- S1・S2 とは独立している。3 本の中では最も小さい
- `mycc.py` は書き換えない。ラッパー `semcc.py`（完成品）が `Sub` の生成だけ差し替える
- 編集するのは `ptrdiff.py` の 2 つの関数

2 コマ 10 でやり残したこと

コマ 10 では、ポインタと整数の演算に要素サイズを掛けることを学んだ。

```
int *p;  
p + 2;      /* 2 * sizeof(int) = 8 バイト進む */  
p - 1;      /* 4 バイト戻る */
```

いまの実装は、この「ポインタ ± 整数」を正しく処理している。しかし C には、もう 1 つのポインタ演算がある。

```
int *p;  
int *q;  
q - p;      /* ポインタ同士の引き算 */
```

これはアドレスの差ではなく、要素いくつつ分離れているかを返す規則になっている。

```
int a[10];
int *p = a;
int *q = &a[7];

q - p;      /* C の答え: 7 (要素 7 個分) */
```

いまの実装は `sub` するだけなので、**28** (バイト差) を返してしまう。

重要

この誤りは、S1 (短絡評価) と同じくテストが通ってしまう種類のものである。コマ 10 のテストは `a[i]` と `*(p + 2)` しか書いていないので、`q - p` を書いた人だけが「なぜか 7 倍ずれる」現象に出会う。

3 なぜ「要素数」なのか

一貫性のためである。C のポインタ演算は、次の等式が成り立つように設計されている。

$$(p + n) - p == n$$

`p + n` が「`n` 要素進む」なら、その差は「`n`」でなければ辻褄が合わない。`p + 2` は 8 バイト進むが、`(p + 2) - p` は 2 になる。

この規則のおかげで、次のような書き方が自然にできる。

```
int my_strlen(char *s) {
    char *p = s;
    while (*p != 0) { p = p + 1; }
    return p - s;      /* 進んだ「文字数」がそのまま長さ */
}
```

`char` は 1 バイトなので、この場合はバイト差と要素数が一致する (だから `char*` では誤りが表面化しない)。`int*` や構造体ポインタで初めてずれる。

4 どう直すか

引き算のあとに、要素サイズで割る。

左辺を評価して退避

右辺を評価

```
sub a0, a1, a0      ← アドレスの差（バイト数）
li  a1, 4           ← 要素サイズ
div a0, a0, a1      ← 要素数に直す
```

要素サイズが 1 (char*) のときは、割り算を出す必要がない。

判定も要る。Sub ノードは 3 種類が同居している。

式	型	生成
<code>p - q</code>	ポインタ - ポインタ	今回追加: 引いてから割る
<code>p - 3</code>	ポインタ - 整数	既存: 3 に要素サイズを掛けてから引く
<code>i - j</code>	整数 - 整数	既存: そのまま引く

左右の型を `cg._type_of_expr()` で調べ、両方がポインタのときだけ新しい生成に回す。

補足

結果の型も変わる。`p - 3` の型はポインタ (int*) だが、`p - q` の型は整数である。そうでないと `(q - p) + 1` がポインタ演算になり、1 ではなく 4 足されてしまう。この型の修正はラッパー `semcc.py` の側で行っている (`_type_of_expr` の差し替え)。本物の C では、この型に `ptrdiff_t` という名前が付いている。

5 実装

Step	実装対象	内容
1	<code>is_ptr_diff</code>	両辺がポインタの Sub かを判定する
2	<code>gen_ptr_diff</code>	引いてから要素サイズで割る

生成の骨格 (退避 → 評価 → 復元 → 演算) は、コマ 3 以来の二項演算パターンそのままである。

```
python3 semcc.py tests/count.c
python3 semcc.py tests/count.c | grep -A2 sub
```

6 テスト

```
python3 check.py      # 判定と生成命令列を確認（未実装は SKIP）
python3 golden.py     # 修正なしで落ち、修正ありで通ること + fixed15
```

テスト	確認内容
count.c	&a[7] - a が 7 になる（28 ではない）
char_diff.c	char* の差（要素サイズ 1）が壊れていない
use_result.c	差の結果が int として扱われる ((q-p)*2、(q-p)+1)

7 発展課題

1. ポインタの比較: `p < q` や `p == q` は正しく動くか確かめる（アドレスの大小比較なので、要素サイズは関係ない）
2. 負の差: `p - q` で `p < q` のとき、結果は負になる。`div` は符号付き除算なので正しく動くはずだが、実際に確かめる
3. 配列の要素数を求める: `sizeof(a) / sizeof(a[0])` という定番の書き方がいまのコンパイラで動くか試す。動かないなら何が足りないか
4. 異なる型のポインタの差: `int *p; char *s; p - s;` は C ではコンパイルエラーである。Q1 の型検査に「ポインタ同士の引き算は同じ型でなければならない」を追加する

補足

コラム: なぜ `div` が要るのか。 要素サイズが 2 の冪（4 や 8）なら、割り算ではなく右シフト（`srai`）で済む。除算命令は乗算より数倍遅いので、実際のコンパイラは「サイズが 2 の冪ならシフト」という最適化を必ず行っている。B1 のピープホール最適化の練習

問題として、`li a1, 4 + div a0, a0, a1` を `srai a0, a0, 2` に置き換える規則を書いてみるとよい。